CS482 Software Project:
A Web-Based Multiplayer Texas Holdem Poker Application

Finn, Chris, Royal, Kelechi

2026-11-06

1 Client Information

- **Client name:** Eric Cui
- **Client title:** Loyola University Maryland Computer Science Professor
- **Client email address:** lcui@loyola.edu
- **Client employer:** Loyola University Maryland
- **How we know the client:** He has taught members of our group in the Computer Science Department.

2 Project Description

2.1 Overview

Our client wants a browser Texas Holdem poker app that users can play with friends. The application will use virtual chips only and will focus on casual play, learning, and competition rather than real-money gambling.

Users will be able to create accounts, browse poker tables, create their own tables, and play live Texas Holdem against other players. The application will also support computer-controlled players so that games can still be played when there are not enough human players online.

Registered users will have virtual chip balances, statistics, friends, messages, and leaderboard rankings. The application will also include a rules page and other tools for players who are learning poker. Administrators will be able to manage users and poker tables.

2.2 Key Features

The main features of the application include:

- User registration, login, logout, and guest access
- Texas Holdem gameplay
- Real-time multiplayer games
- Public poker tables and table creation
- Virtual chip balances and buy-ins
- Player statistics and leaderboards
- Friends and messaging
- Poker rules and optional hints

- Computer-controlled poker players
- Administrator controls

Some lower-priority features may be added later if time allows. These include private tables, tournaments, achievements, seasonal leaderboards, different themes, spectator mode, adjustable bot difficulty, and an external bot API.

2.3 Why this Project is Interesting

This project combines several areas of computer science and software engineering. The largest challenge is keeping multiple players synchronized during a live poker game. The server must also correctly enforce poker rules, calculate winners, update chip balances, and prevent users from performing illegal actions.

The project also gives our group experience with web development, databases, networking, testing, software architecture, and team-based development.

2.4 Areas of Computer Science Required

This project will use concepts from several areas of computer science:

- Software engineering
- Web development
- Databases
- Networking
- Algorithms
- Artificial intelligence
- Computer security
- Human-computer interaction

2.5 Potential Concerns

Our main concern is project scope. Real-time multiplayer poker already requires a large amount of game logic and testing, so the team will focus on the high-priority requirements first.

Another concern is preventing cheating. The browser should never be trusted to directly control the game. The server will validate player actions and remain responsible for the official game state.

Finally, some members of the team have limited experience with real-time web communication. We plan to reduce this risk by developing and testing the basic poker engine before integrating the complete multiplayer system.

3 Requirements

The revised User Stories from the previous assignment will be used as the functional requirements for the project. The stories are organized by priority so that the team can focus first on the functionality required to produce a working multiplayer poker game.

The highest-priority requirements include user accounts, correct Texas Holdem gameplay, public poker tables, live multiplayer updates, virtual chip balances, leaderboards, inactivity handling, and administrator controls.

The project also contains several non-functional requirements. These include security, authorization, performance, recovery from short disconnections, scalability, data integrity, privacy, anti-cheat protection, usability, browser compatibility, and maintainability.

4 System Design

4.1 Architecture

Our team will use a **client-server architecture** organized using a **three-layer architecture**. The user's browser will act as the client and communicate with the backend server.

The three main layers are:

Presentation Layer Contains the browser interface that the user interacts with. This includes login, the poker lobby, the poker table, friends, messages, statistics, and administration screens.

Application Layer Contains the main business logic of the application. This includes authentication, poker rules, table management, multiplayer communication, bots, statistics, and administration.

Data Layer Handles the storage and retrieval of persistent information such as users, poker tables, games, chip balances, friendships, and messages.

Separating the application into layers should reduce coupling between different parts of the system. For example, the user interface will not directly modify the database or poker game state. Instead, it will send a request to the backend, which will determine whether the action is valid.

The server will be the main source of truth during poker games. This is important because players should not be able to change their cards, chip balances, or available actions from the browser.

The Presentation Layer will communicate with the Application Layer through normal HTTP requests and WebSockets. The Django backend will communicate with PostgreSQL through the Django ORM.

Figure 1 shows the planned architecture.

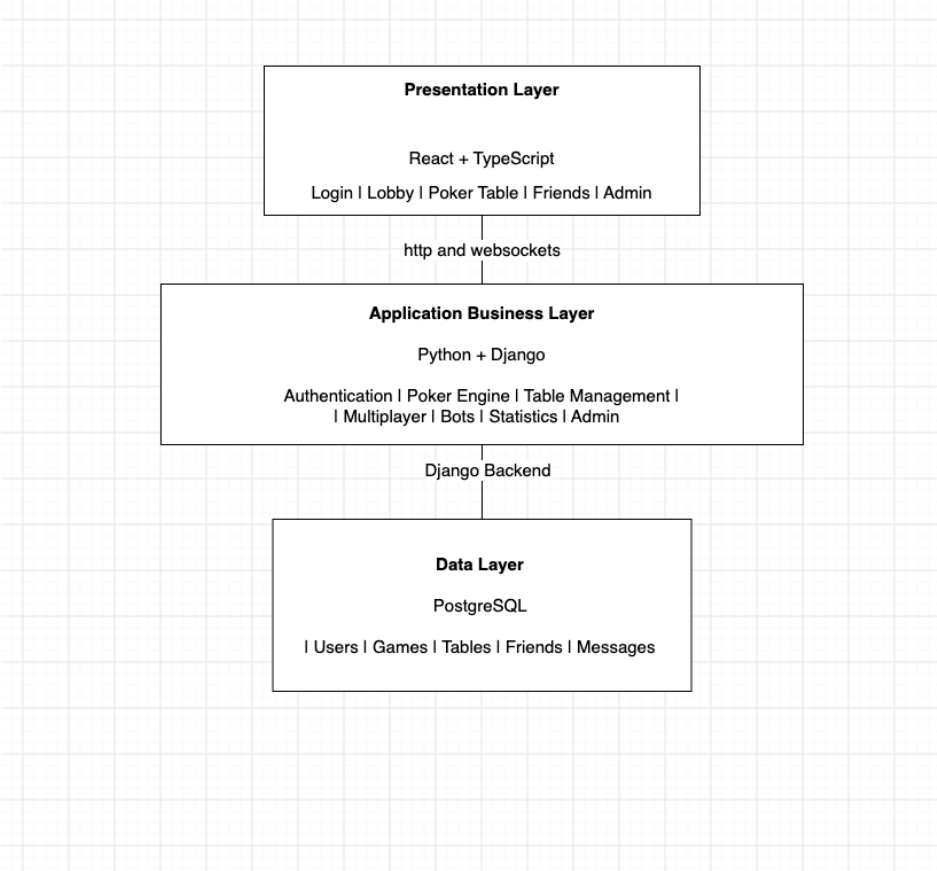


Figure 1: Client-server and three-layer architecture of the poker application.

4.1.1 Main Software Modules

The project will be divided into several main modules:

- **User Management:** Handles registration, login, profiles, avatars, and user roles.
- **Poker Game Engine:** Handles dealing cards, betting, turns, blinds, hand evaluation, pots, and determining the winner.
- **Lobby and Table Management:** Handles creating, browsing, joining, and leaving poker tables.
- **Multiplayer Communication:** Keeps connected players synchronized during live games.
- **Social Features:** Handles friends, online status, messages, invitations, and table chat.
- **Statistics and Leaderboard:** Handles player statistics and leaderboard rankings.
- **Bots:** Handles computer-controlled poker players and practice games.
- **Administration:** Handles moderation of users, reports, and poker tables.

These modules group related responsibilities together. This should help keep the system organized and maintain high cohesion within each module.

4.2 Technology

The frontend of the application will use **React with TypeScript**. React will be used to create the browser interface, while TypeScript will provide type checking and help make the frontend code easier to maintain.

The backend will use **Python with Django**. Django will handle authentication, application logic, database access, and communication with the frontend. Django REST Framework will be used for normal HTTP requests.

Live poker games require information to be updated without refreshing the page. For this reason, the application will use **WebSockets** with **Django Channels** for real-time game communication.

The main database will be **PostgreSQL**. PostgreSQL is a relational database and is appropriate because the project contains many related types of information, including users, games, poker tables, friendships, and messages.

For testing, the backend will use **pytest** and **pytest-django**. The frontend will use **Vitest**. Git and GitHub will be used for version control and collaboration.

4.3 Coding Standards

The team will follow consistent coding standards throughout the project.

Python code will follow the PEP 8 style guide. Python variables and functions will use `snake_case`, while classes will use `PascalCase`. TypeScript variables and functions will use `camelCase`, while React components will use `PascalCase`. Database fields will use lowercase `snake_case` names.

Classes, functions, and modules should have a clear purpose. If one part of the code begins handling too many unrelated responsibilities, it should be separated into smaller parts.

Major features will be developed using separate Git branches. The team will avoid committing unfinished major changes directly to the `main` branch.

Important application logic will have automated unit tests. The team will target at least **60% test coverage**, with additional testing for important parts of the poker game engine. The automated tests should pass before major changes are merged into the main branch.

If a significant bug is found, the team will add a test for the bug when possible so that the same problem can be detected if it occurs again.

4.4 Data

The application will use a relational database. The main planned tables are described below.

Table	**Main Data**
User	Username, email, password hash, avatar, role, virtual chip balance, and account status.
PokerTable	Table name, host, privacy setting, maximum players, buy-in, blinds, and table status.
Game	Poker table, start time, end time, and game status.
Player	User, game, seat position, starting chips, ending chips, and player status.
Hand	Game, hand number, community cards, pot, and winning participant.
Friendship	Requesting user, receiving user, friendship status, and creation date.
Message	Sender, receiver, message text, timestamp, and read status.
Report	åReporting user, reported user or table, reason, details, and report status.

Table 1: Main planned database tables.

The `Player` table connects users and games. A game can contain several users, while a user can participate in many games over time. This creates a many-to-many relationship between users and games.

The `Friendship` table also connects two users. It stores who sent the friend request, who received it, and the current status of the request.

The `Message` table stores the sender and receiver of each private message. The `Report` table allows a user to report another user or a poker table for administrator review.

The exact database design may change as development begins, but these tables represent the main information needed for the current User Stories.

Figure 2 shows the planned relationships between the main tables.

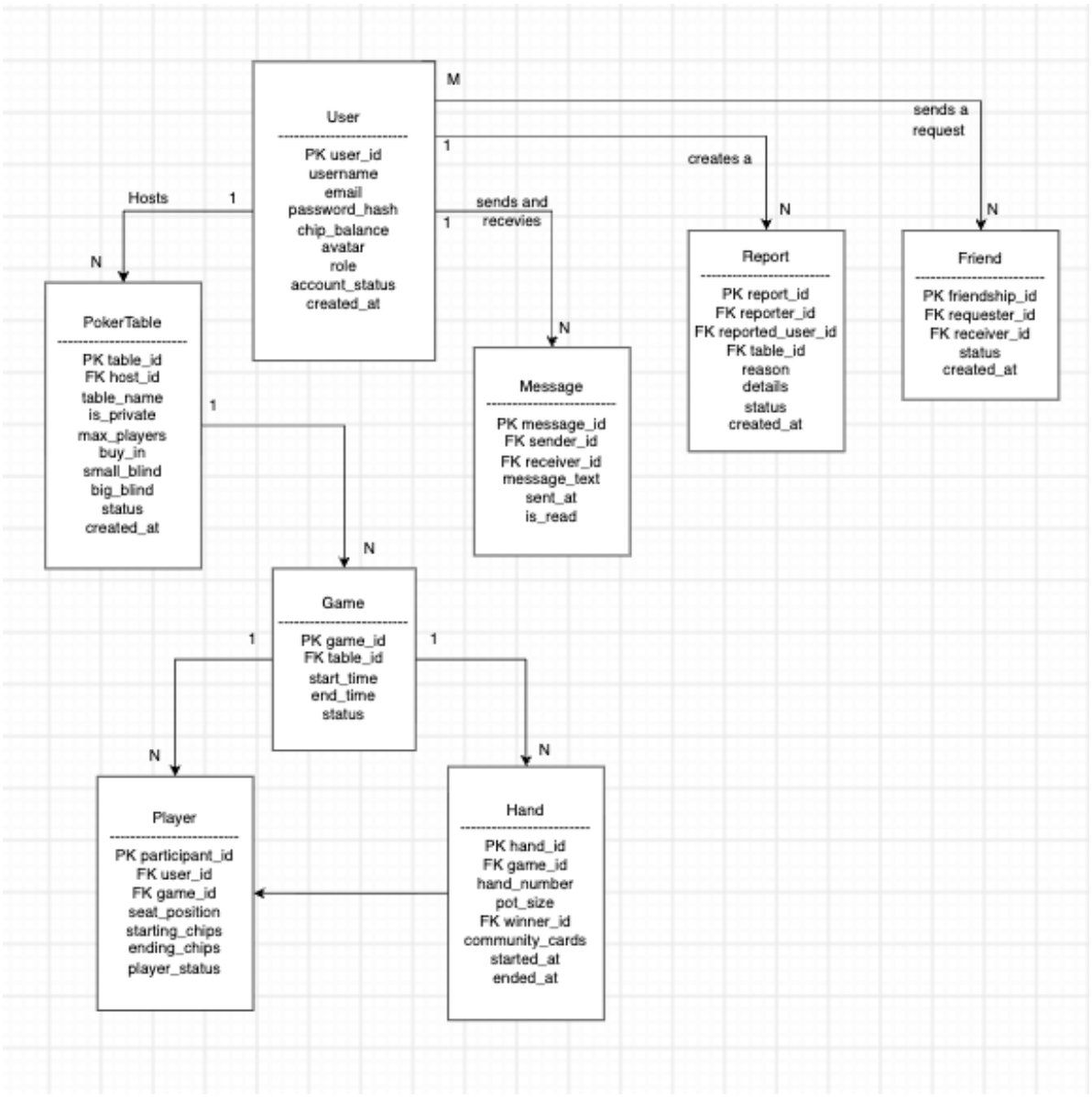


Figure 2: Entity Relationship diagram for the poker application.

4.5 UI Mocks

The user interface mockups were created in Figma. The goal of the mockups is to define the main screens and forms before implementation begins.

The interface uses a consistent layout and color scheme throughout the application. The following figures show several of the main interfaces.

4.5.1 Registration

The registration form allows new players to create an account using an email address, username, and password. Users also confirm their password and age requirement before creating an account, while guest access is available without registration.

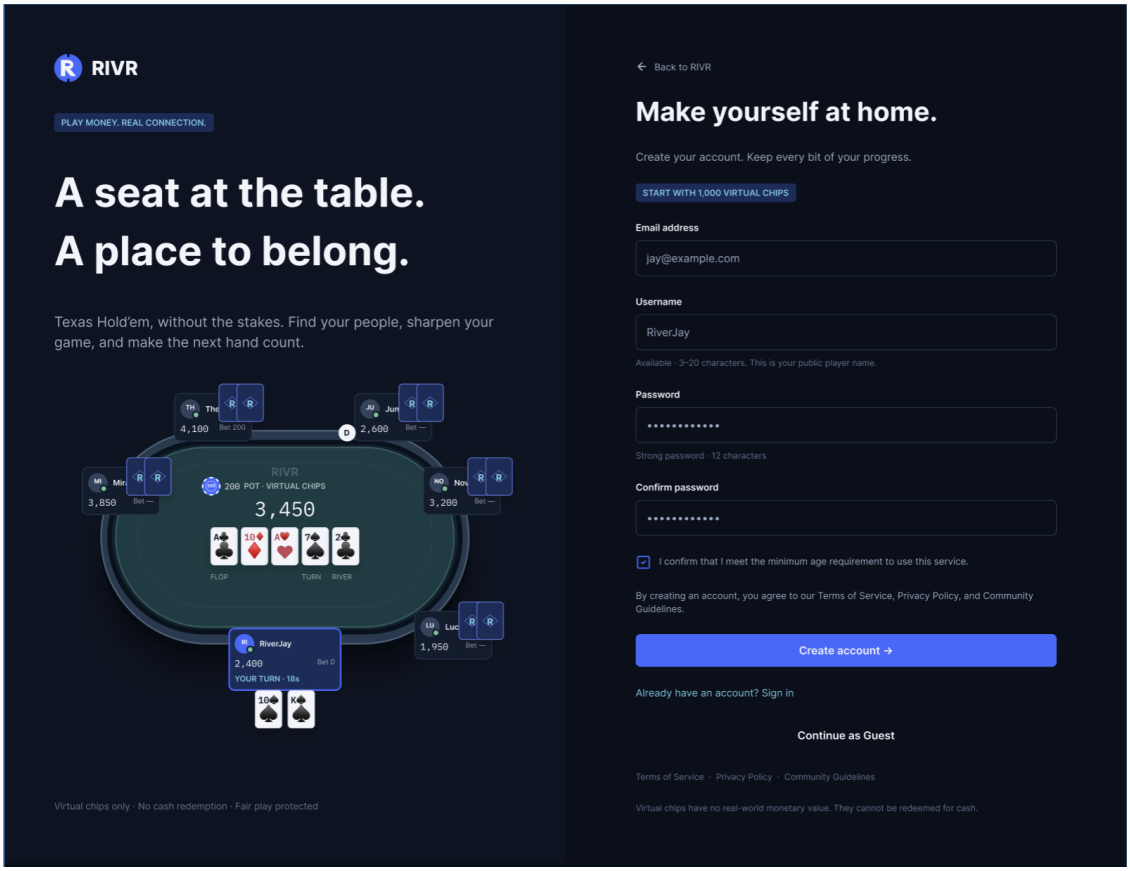


Figure 3: User registration form.

4.5.2 Home

The Home page acts as the main dashboard after login. It gives the player quick access to live games, table creation, bot games, player statistics, friends, and leaderboard information.

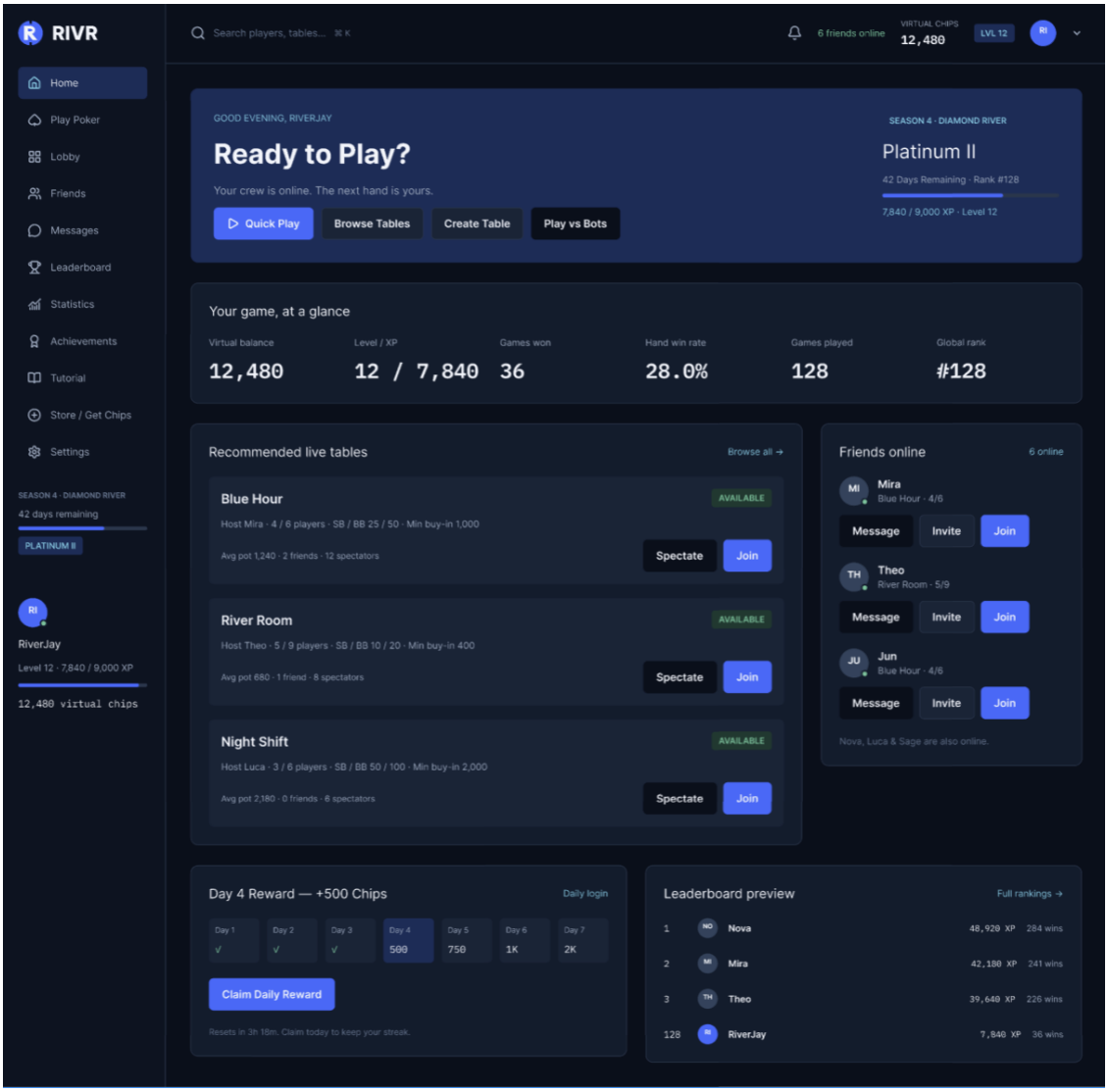


Figure 4: Main player dashboard.

4.5.3 Poker Lobby

The Poker Lobby allows players to view available public poker tables and use filters to find a game. It also shows information such as buy-in, blinds, player count, and table status.

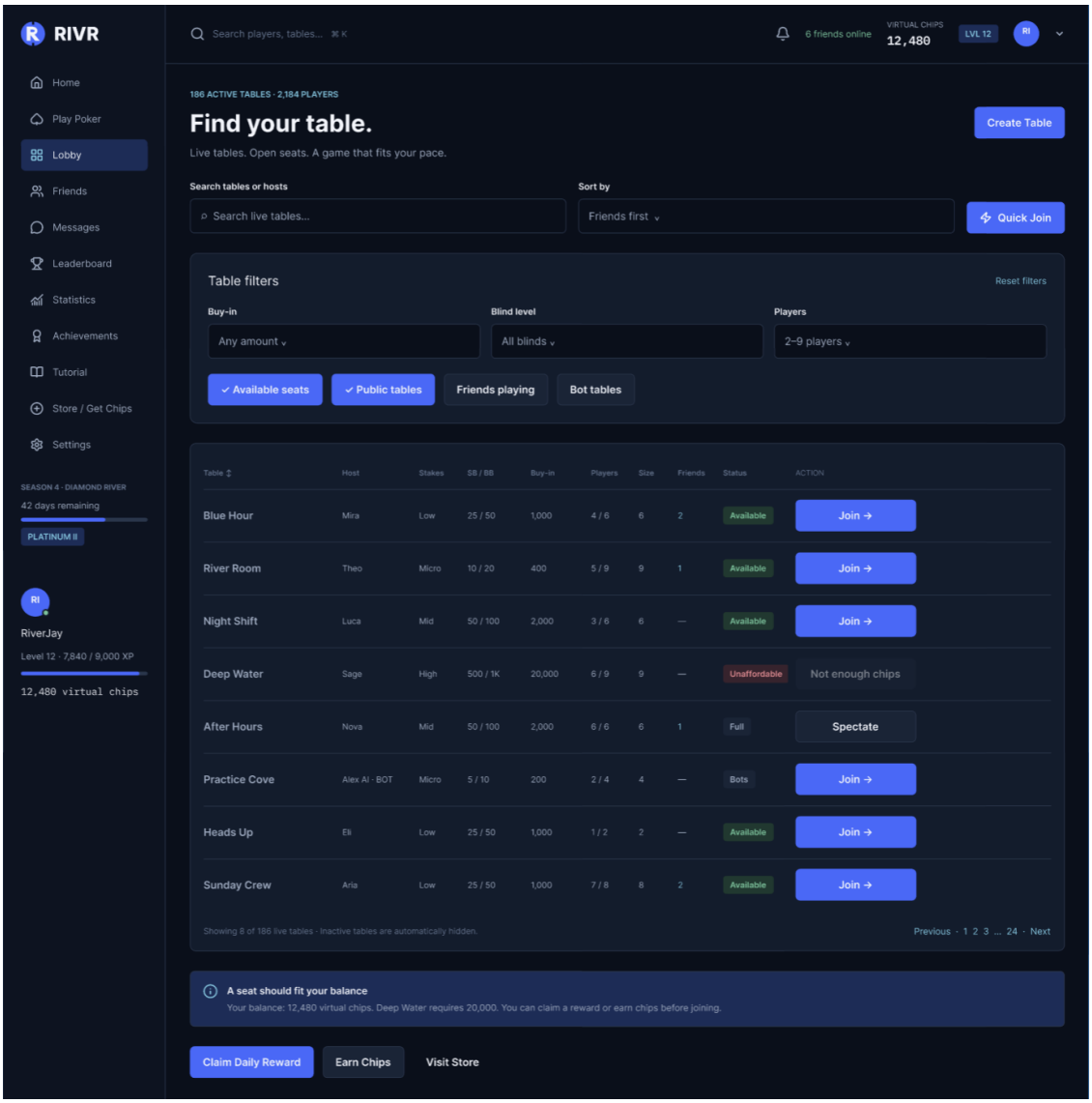


Figure 5: Poker Lobby interface.

4.5.4 Create Table

The Create Table form allows a host to choose settings such as the table name, player limit, buy-in, blinds, privacy, and bot options.

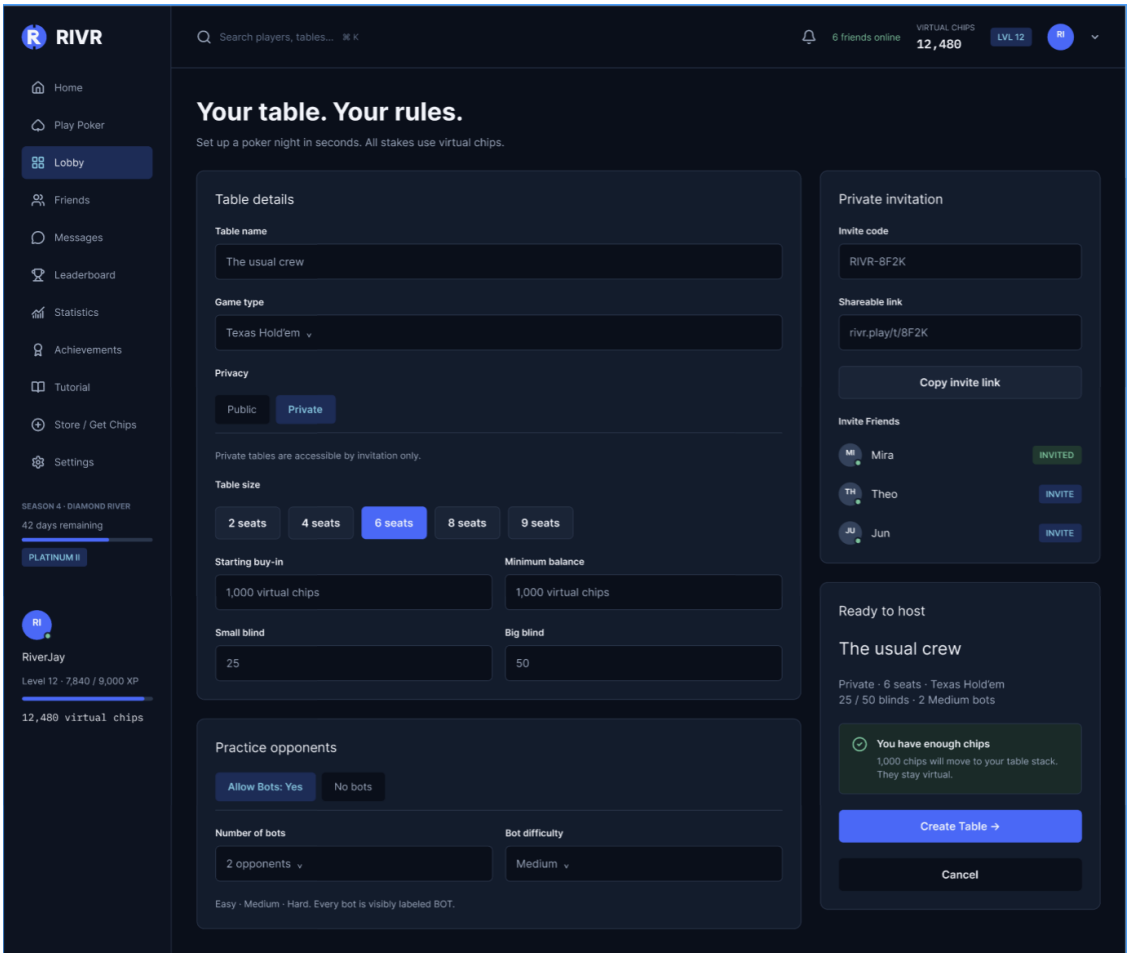


Figure 6: Create Table form.

4.5.5 Poker Game

The main Poker Game interface displays the players, cards, chip stacks, community cards, pot, and available poker actions such as fold, check, call, and raise.

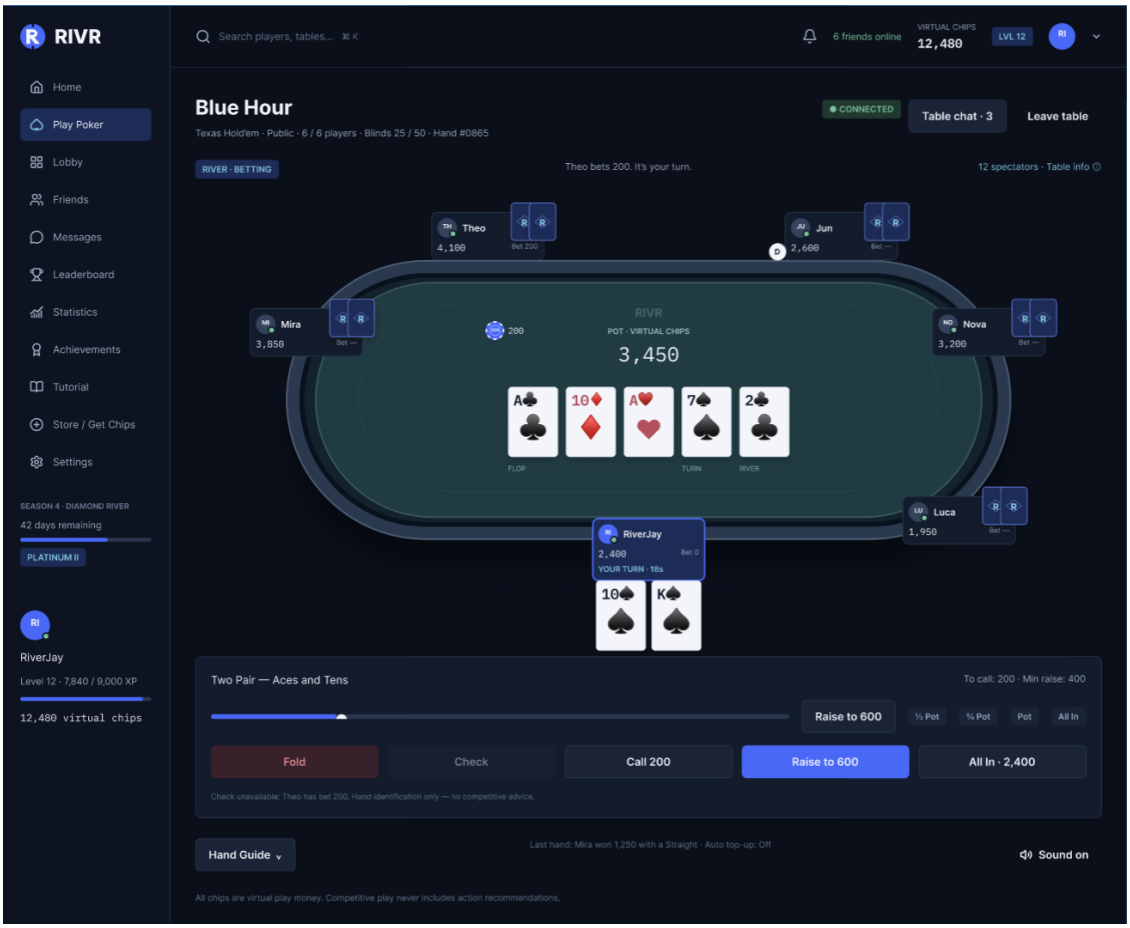


Figure 7: Main Texas Hold'em game interface.

4.5.6 Friends

The Friends page allows registered users to view friends, incoming friend requests, recently played users, and online status. Users can also add or remove friends and invite friends to a poker table.

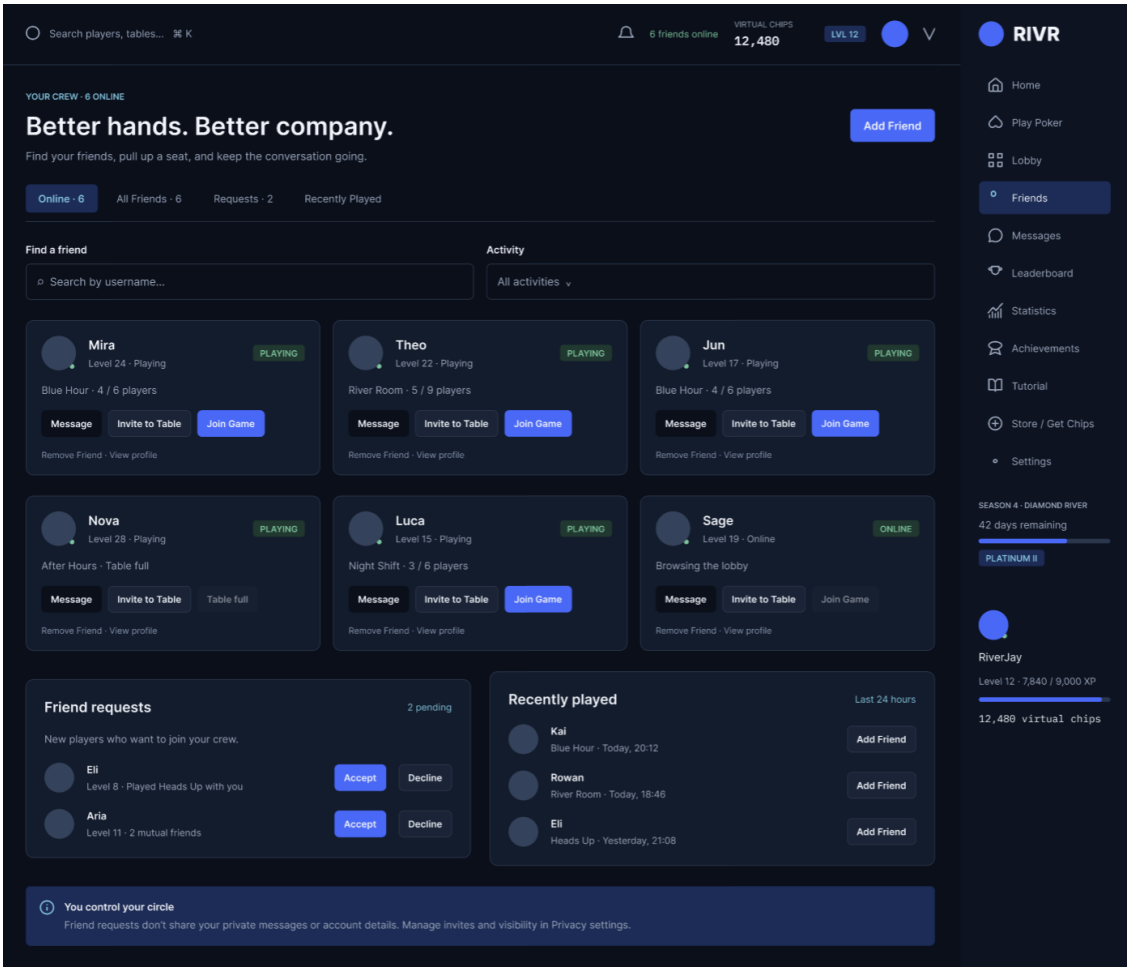


Figure 8: Friends interface.

4.5.7 Messages

The Messages interface allows players to send private messages to friends and share poker table invitations. Table chat is also available from the main poker game interface.

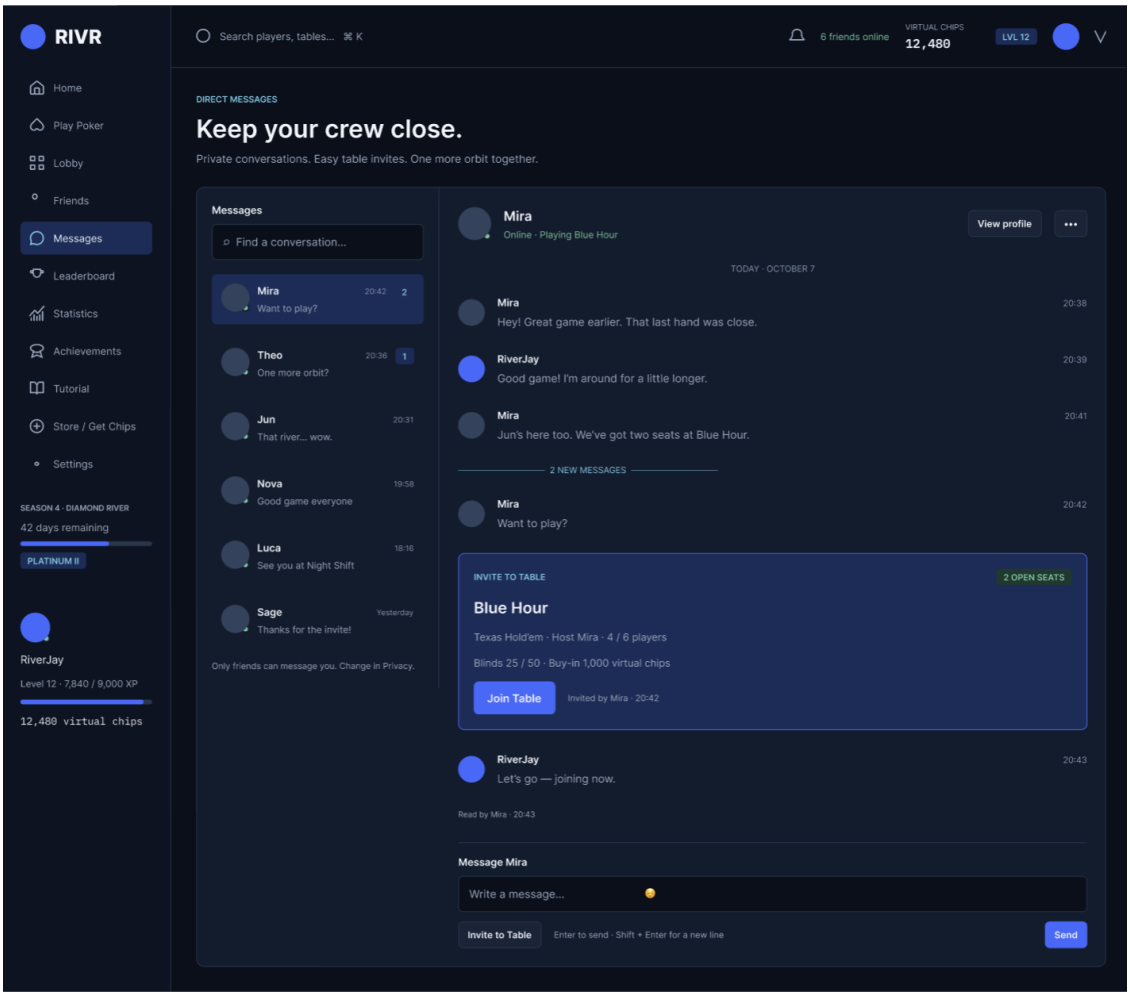


Figure 9: Private messaging interface.

4.5.8 Settings and Profile

The Settings interface allows a registered user to update account information, appearance, privacy settings, notifications, and game preferences. Users can also change their password or delete their account.

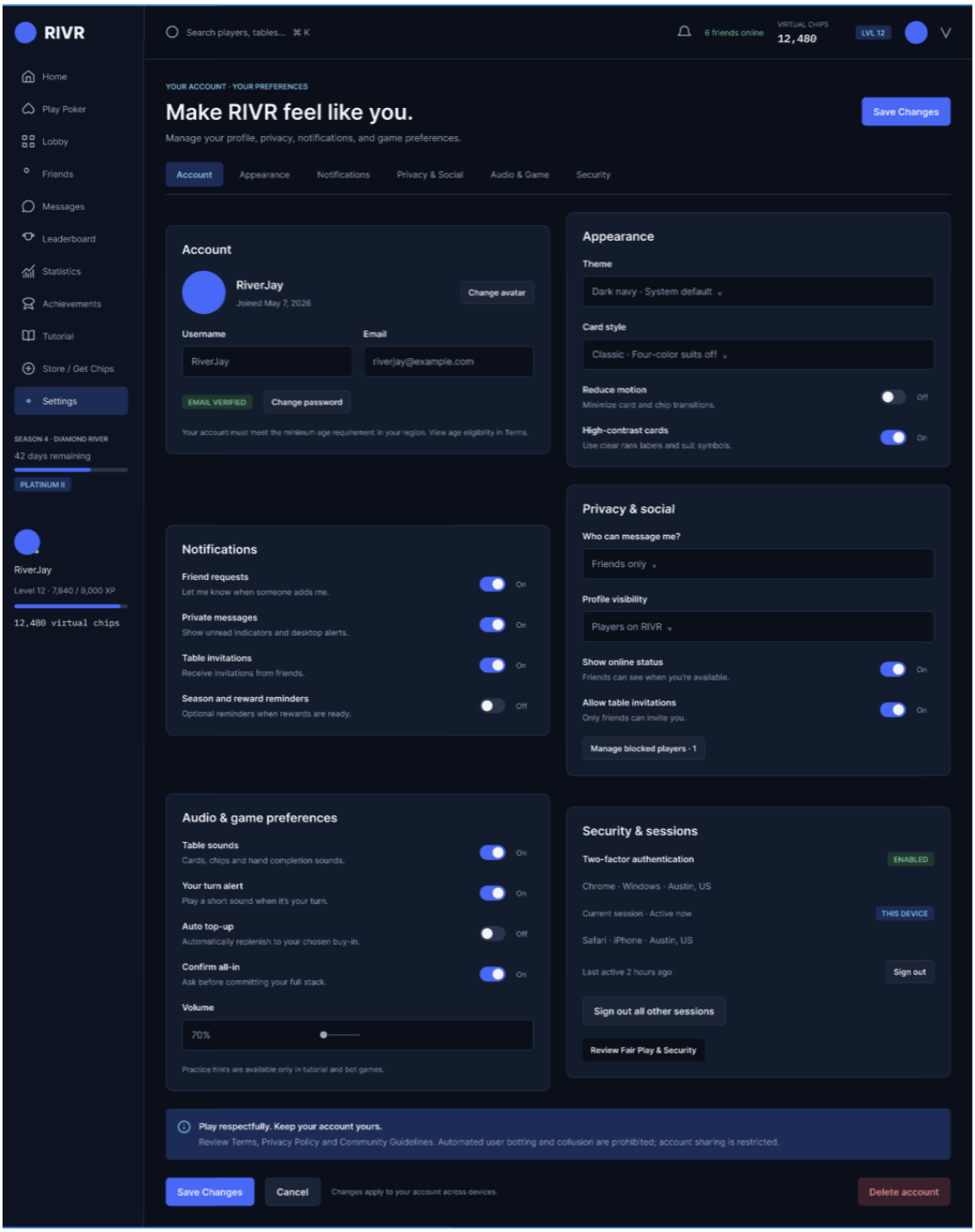


Figure 10: User settings and account interface.

4.5.9 Administrator Dashboard

The Administrator Dashboard allows administrators to view active users, active tables, reports, and suspicious activity. Administrators can perform moderation actions such as removing users from games, suspending or banning accounts, and closing tables.

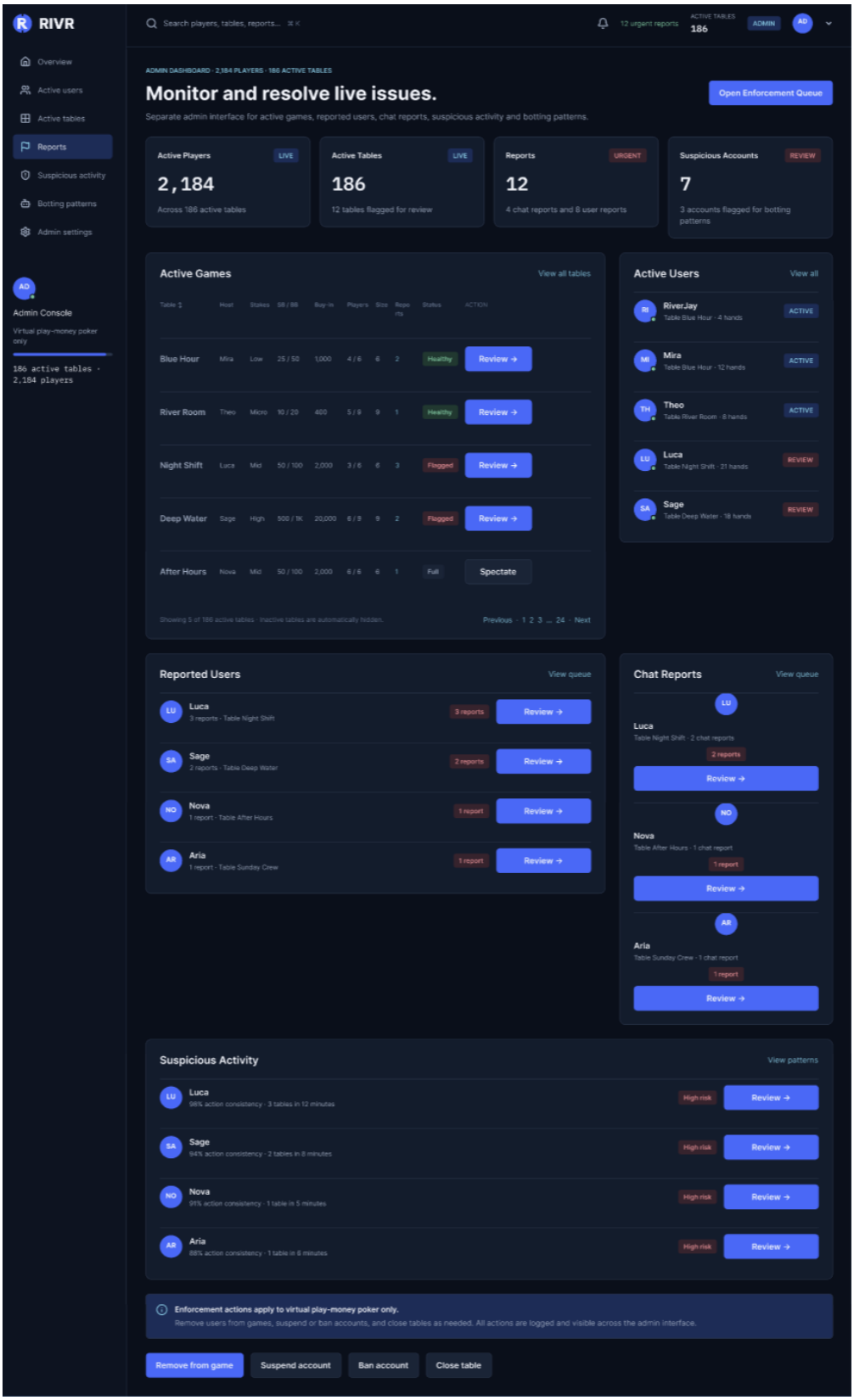


Figure 11: Administrator dashboard.

4.5.10 Forms and CRUD Interfaces

The application contains several major forms, including registration, login, Create Table, settings, messaging, and report forms.

User accounts provide the clearest example of CRUD functionality. A user account is **created** through registration, **read** through the profile page, **updated** through the Settings page, and **deleted** through account settings.

Friend management provides another example of data management in the interface. Users can send friend requests, view existing friends and incoming requests, accept or decline requests, and remove friends.

The Poker Lobby and Create Table screens provide the main interfaces for viewing and creating poker tables. Administrators also have controls for managing and closing tables when necessary.

4.6 Iteration/Sprint 1

4.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

4.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

4.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

Figure 12 shows the test coverage

4.6.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

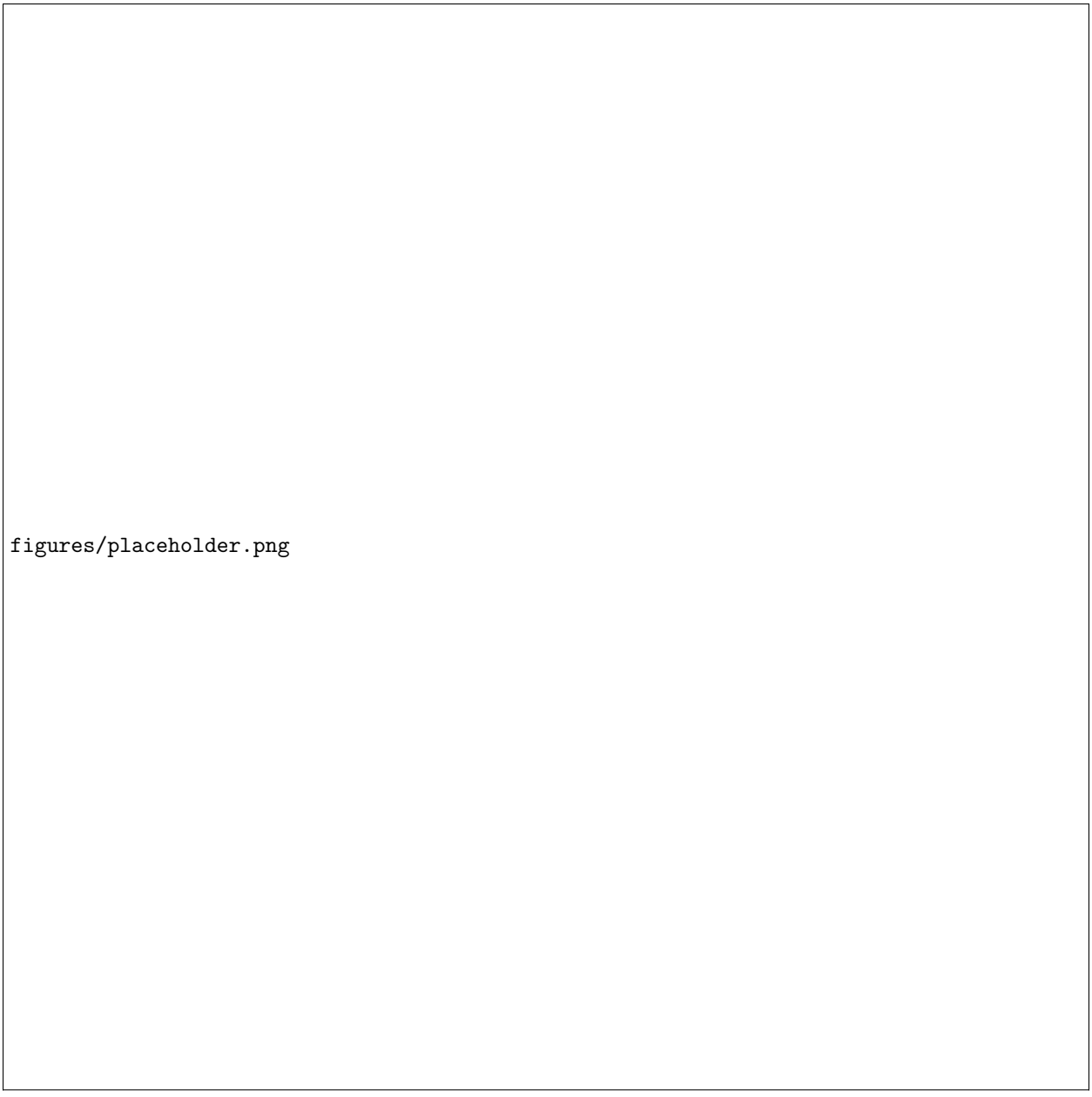


Figure 12: Iteration 1 test coverage report